

# Java AWT Tutorial

Abstract Window Toolkit – Complete Study Guide

Components • Containers • Event Handling • Exam Examples

■ Exam Ready

■ Full Theory

■ Code Examples

## Table of Contents

1. What is Java AWT?
2. AWT Architecture & Hierarchy
3. Types of Containers
4. AWT Components (Label, Button, TextField, Checkbox ...)
5. Event Handling (ActionListener, MouseListener, KeyListener ...)
6. Adapter Classes
7. Code Examples
8. Exam Practice Questions

# 1. What is Java AWT?

Java AWT (Abstract Window Toolkit) is an API used to create Graphical User Interface (GUI) or window-based Java programs. AWT components are **platform-dependent** — they are rendered using the native OS controls, so a button on Windows looks like a Windows button and on Mac it looks like a Mac button.

## Key Characteristics

|                    |                                                                        |
|--------------------|------------------------------------------------------------------------|
| Heavy-weight       | Components use OS-level resources directly.                            |
| Platform-dependent | Visual appearance differs across OS (Windows, Mac, Linux).             |
| Part of JFC        | AWT is part of the Java Foundation Classes.                            |
| Package            | java.awt contains all AWT classes.                                     |
| Abstract API       | Java code talks to AWT's abstract layer; AWT calls native OS routines. |

## Why AWT is Still Platform-Independent?

- **JVM:** Your Java bytecode runs on any OS via the JVM — the JVM handles OS translation.
- **Abstract APIs:** You write to AWT's abstract interface; AWT internally calls OS-specific code.
- **Platform-Independent Libraries:** AWT's own library code is written in Java, ensuring consistent behaviour.

■ **Note:** *AWT's components look different on each OS, but your Java code stays the same — that's the platform independence.*

## 2. AWT Architecture & Hierarchy

|                  |                                                                                               |
|------------------|-----------------------------------------------------------------------------------------------|
| Component        | Base class for all AWT visual elements (Button, Label, TextField, etc.)                       |
| Container        | A Component that can hold other Components (Panel, Frame, Dialog, Window).                    |
| Layout Manager   | Controls arrangement of components inside a container (BorderLayout, FlowLayout, GridLayout). |
| Event Handling   | Mechanism to respond to user actions — uses Listeners and Adapters.                           |
| Graphics/Drawing | Tools to draw shapes, insert images, and write text directly on components.                   |

■ **Note:** A Container is itself a type of Component, so you can nest containers inside containers.

### 3. Types of Containers in Java AWT

#### Window

Top-level container representing a graphical window or dialog. Extends Container so it can hold other components. No title bar by default.

#### Panel

Lightweight, invisible container used to group components inside a Frame. Works like a div in HTML.

#### Frame

Most commonly used container. Has a title bar, border, and can hold menu bars — your typical desktop app window.

#### Dialog

A temporary popup window used to get input from the user (e.g., Save File dialog, Alert box).

## 4. AWT Components

### Label

Displays static, non-editable text on screen.

#### Syntax:

```
public class Label extends Component implements Accessible
```

#### Constructors:

- `Label()` — empty label
- `Label(String str)` — label with text
- `Label(String str, int alignment)` — aligned text

### Button

A clickable button that generates an `ActionEvent` when clicked.

#### Syntax:

```
public class Button extends Component implements Accessible
```

#### Constructors:

- `Button()` — empty button (shows blank box)
- `Button(String str)` — button with label text

### TextField

Single-line text input box where users can type.

#### Syntax:

```
public class TextField extends TextComponent
```

#### Constructors:

- `TextField()` — empty box
- `TextField(String text)` — pre-filled text
- `TextField(int col)` — specified width in columns
- `TextField(String str, int col)` — pre-filled with width

## Checkbox

A toggleable box (checked/unchecked). Groups of checkboxes act as radio buttons via CheckboxGroup.

### Syntax:

```
public class Checkbox extends Component implements ItemSelectable, Accessible
```

### Constructors:

- Checkbox() — no label
- Checkbox(String str) — with label
- Checkbox(String str, boolean state, CheckboxGroup grp) — with group

## CheckboxGroup

Groups Checkboxes so only ONE can be selected at a time — this is how radio buttons work in AWT.

### Syntax:

```
public class CheckboxGroup extends Object implements Serializable
```

### Constructors:

- CheckboxGroup() — creates a group

## Choice

A drop-down popup menu of choices (like HTML select element).

### Syntax:

```
public class Choice extends Component implements ItemSelectable, Accessible
```

### Constructors:

- Choice() — creates a new choice menu

## List

A scrollable list of text items; multiple items can be visible simultaneously.

### Syntax:

```
public class List extends Component implements ItemSelectable, Accessible
```

### Constructors:

- List() — new list
- List(int row) — list with given number of visible rows
- List(int row, boolean mode) — list with multi-select mode

## Canvas

A blank rectangular area you can draw on — shapes, images, custom graphics.

### Syntax:

```
public class Canvas extends Component implements Accessible
```

### Constructors:

- `Canvas()` — new blank canvas
- `Canvas(GraphicsConfiguration config)` — with config

## Scrollbar

A slider bar for scrolling content, either vertical or horizontal.

### Syntax:

```
public class Scrollbar extends Component implements Adjustable, Accessible
```

### Constructors:

- `Scrollbar()` — vertical scrollbar
- `Scrollbar(int orientation)` — vertical or horizontal
- `Scrollbar(int orientation, int value, int visible, int min, int max)`

## MenuItem & Menu

Menu is a dropdown (File, Edit); MenuItem is each option inside (Save, Exit).

### Syntax:

```
public class Menu extends MenuItem implements MenuContainer, Accessible
```

### Constructors:

- `MenuItem` — individual menu option
- `Menu` — dropdown containing MenuItems
- `PopupMenu` — context menu on right-click

## Panel

A lightweight container to hold and organize components within a Frame.

### Syntax:

```
public class Panel extends Container implements Accessible
```

### Constructors:

- `Panel()` — uses FlowLayout by default

## Toolkit

Utility class providing platform-independent access to system resources (screen size, system fonts, etc.).

### Syntax:

```
public abstract class Toolkit extends Object
```

### Constructors:

- `Toolkit.getDefaultToolkit()` — get system toolkit



## 5. Event Handling in Java AWT

Events are user actions like button clicks, key presses, or window operations. AWT uses **Listener Interfaces** — you implement the interface and define what happens when the event fires.

### ActionListener

Responds to button clicks, menu selections, and text field Enter key.

#### Syntax:

```
public class MyClass implements ActionListener
```

#### Methods:

- actionPerformed(ActionEvent e) — fires when action occurs

### MouseListener

Responds to mouse button events on a component.

#### Syntax:

```
public class MyClass implements MouseListener
```

#### Methods:

- mouseClicked(MouseEvent e) — mouse button clicked
- mousePressed(MouseEvent e) — mouse button pressed/held
- mouseReleased(MouseEvent e) — mouse button released
- mouseEntered(MouseEvent e) — cursor enters component
- mouseExited(MouseEvent e) — cursor leaves component

### MouseMotionListener

Responds to mouse movement — moving or dragging.

#### Syntax:

```
public class MyClass implements MouseMotionListener
```

#### Methods:

- mouseMoved(MouseEvent e) — cursor moves (no button held)
- mouseDragged(MouseEvent e) — cursor moves with button held

### ItemListener

Responds to selection/deselection of Checkboxes, RadioButtons, List items.

#### Syntax:

```
public class MyClass implements ItemListener
```

#### Methods:

- itemStateChanged(ItemEvent e) — fires on selection change

## KeyListener

Responds to keyboard events.

### Syntax:

```
public class MyClass implements KeyListener
```

### Methods:

- keyPressed(KeyEvent e) — key is held down
- keyReleased(KeyEvent e) — key is released
- keyTyped(KeyEvent e) — key character is typed (pressed + released)

## WindowListener

Responds to window-level events (open, close, minimize, restore).

### Syntax:

```
public interface WindowListener extends EventListener
```

### Methods:

- windowOpened(WindowEvent e) — window first appears
- windowClosing(WindowEvent e) — user hits the X button
- windowClosed(WindowEvent e) — window fully closed
- windowIconified(WindowEvent e) — window minimized
- windowDeiconified(WindowEvent e) — window restored
- windowActivated(WindowEvent e) — window gains focus
- windowDeactivated(WindowEvent e) — window loses focus

## Adapter Classes

Adapter classes provide **empty default implementations** of all listener interface methods. Instead of implementing every method in WindowListener (7 methods!), extend **WindowAdapter** and override only the ones you need.

|                    |                                               |
|--------------------|-----------------------------------------------|
| WindowAdapter      | Default implementation of WindowListener      |
| MouseAdapter       | Default implementation of MouseListener       |
| MouseMotionAdapter | Default implementation of MouseMotionListener |
| KeyAdapter         | Default implementation of KeyListener         |

■ **Note:** *dispose()* releases OS resources and removes the window from screen. Use it to close a window without exiting the entire program.

## 6. Code Examples

### Example 1 — Hello World in AWT

Creates a Frame window and displays a centered 'Hello World!' label.

```
import java.awt.*;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

public class AWT_Example {
    public static void main(String[] args) {
        Frame frame = new Frame("Basic Program"); // Create Frame
        Label label = new Label("Hello World!"); // Create Label

        label.setAlignment(Label.CENTER);          // Centre-align
        frame.add(label);                          // Add label to frame
        frame.setSize(300, 300);                   // Set window size
        frame.setVisible(true);                    // Show window

        // Close window on X button click
        frame.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
    }
}
```

### Example 2 — AWT Button

Creates a Frame with a Button positioned at specific coordinates using `setBounds()`.

```
import java.awt.*;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

public class Button_Example {
    public static void main(String[] args) {
        Frame frame = new Frame("Example 2");
        Button button = new Button("Click Here");

        // setBounds(x, y, width, height)
        button.setBounds(80, 100, 64, 30);

        frame.add(button);
        frame.setSize(300, 300);
    }
}
```

```

        frame.setLayout(null);          // Disable auto layout
        frame.setVisible(true);

        frame.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
    }
}

```

## Example 3 — ActionListener on Button

Handles button click events and prints a message to the console.

```

import java.awt.*;
import java.awt.event.*;

public class ActionExample implements ActionListener {
    Frame frame;
    Button btn;
    Label lbl;

    ActionExample() {
        frame = new Frame("Action Listener Demo");
        btn = new Button("Click Me");
        lbl = new Label("Status: Waiting...");

        btn.setBounds(100, 100, 80, 30);
        lbl.setBounds(60, 160, 200, 30);

        btn.addActionListener(this); // Register listener

        frame.add(btn);
        frame.add(lbl);
        frame.setSize(320, 280);
        frame.setLayout(null);
        frame.setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        lbl.setText("Status: Button Clicked!");
    }

    public static void main(String[] args) {
        new ActionExample();
    }
}

```

```
}
```

## Example 4 — KeyListener

Detects and displays key press events in real time.

```
import java.awt.*;
import java.awt.event.*;

public class KeyExample extends Frame implements KeyListener {
    Label lbl;

    KeyExample() {
        lbl = new Label("Press any key...");
        lbl.setBounds(60, 120, 220, 30);
        add(lbl);

        addKeyListener(this);    // Register KeyListener on the Frame
        setSize(320, 280);
        setLayout(null);
        setVisible(true);
    }

    public void keyPressed(KeyEvent e) {
        lbl.setText("Pressed: " + e.getKeyChar());
    }

    public void keyReleased(KeyEvent e) {
        lbl.setText("Released: " + e.getKeyChar());
    }

    public void keyTyped(KeyEvent e) {}    // Must implement all 3 methods

    public static void main(String[] args) {
        new KeyExample();
    }
}
```

## 7. Exam Practice Questions

The following questions are highly likely to appear in university/college exams on Java AWT. Answers are concise and exam-ready.

### A. Short Answer / Theory Questions

**Q1. What is Java AWT? State its full form.**

AWT stands for Abstract Window Toolkit.

It is an API in Java used to create Graphical User Interface (GUI) applications.

It is part of the Java Foundation Classes (JFC) and uses native OS components for rendering.

**Q2. What are the two main characteristics of AWT components?**

1. Heavy-weight: they consume native OS resources.
2. Platform-dependent: their appearance differs across operating systems.

**Q3. Differentiate between AWT and Swing.**

AWT: platform-dependent, heavy-weight, uses native OS components, older API.

Swing: platform-independent, light-weight, draws its own components, part of JFC.

Swing provides more components and finer control over look-and-feel.

**Q4. List the four types of containers in Java AWT.**

1. Window
2. Panel
3. Frame
4. Dialog

**Q5. What is the difference between a Frame and a Dialog?**

Frame: main application window with title bar, border, and optional menu bar; persistent.

Dialog: a temporary popup window to collect user input; depends on a parent Frame.

**Q6. What is the role of a Layout Manager in AWT?**

A Layout Manager automatically arranges components inside a container.

Examples: FlowLayout (left-to-right), BorderLayout (5 regions), GridLayout (grid of cells).

setLayout(null) disables automatic layout and lets you use setBounds() for manual positioning.

**Q7. What is the difference between mouseClicked and mousePressed?**

mousePressed fires as soon as the mouse button is held down.

mouseClicked fires only after the button is pressed AND released (complete click).

**Q8. What is an Adapter class in AWT? Give one example.**

An Adapter class provides empty default implementations of all methods in a listener interface.

This avoids writing empty methods you don't need.

Example: WindowAdapter implements all 7 methods of WindowListener with empty bodies.

You extend WindowAdapter and override only windowClosing().

#### **Q9. What does dispose() do in AWT?**

dispose() releases all OS resources associated with the window and removes it from the screen.

It is used to close a window without shutting down the entire Java program (unlike System.exit(0)).

#### **Q10. What is CheckboxGroup and why is it used?**

CheckboxGroup groups multiple Checkbox components together.

Only one Checkbox in the group can be selected at a time.

This is how radio buttons are implemented in AWT.

## **B. Multiple Choice Questions (MCQs)**

#### **Q1. Which package contains all AWT classes?**

Options: (a) java.swing (b) java.awt (c) java.gui (d) java.window

Answer: (b) java.awt

#### **Q2. Which container has a title bar and can hold a menu bar?**

Options: (a) Panel (b) Window (c) Frame (d) Dialog

Answer: (c) Frame

#### **Q3. Which method is used to add a component to a container?**

Options: (a) insert() (b) append() (c) add() (d) attach()

Answer: (c) add()

#### **Q4. How many methods does ActionListener have?**

Options: (a) 1 (b) 3 (c) 5 (d) 7

Answer: (a) 1 — only actionPerformed()

#### **Q5. Which listener is used to detect mouse movement WITHOUT clicking?**

Options: (a) MouseListener (b) MouseMotionListener (c) ActionListener (d) ItemListener

Answer: (b) MouseMotionListener — mouseMoved() method

#### **Q6. Which method in WindowListener is called when the user clicks the X button?**

Options: (a) windowClosed() (b) windowClosing() (c) windowDeactivated() (d) windowExit()

Answer: (b) windowClosing()

**Q7. setLayout(null) in a Frame means:**

Options: (a) No components can be added (b) Components arrange automatically  
(c) Manual positioning via setBounds() (d) The frame becomes invisible

Answer: (c) Manual positioning via setBounds()

**Q8. Which AWT component is used to create a drop-down menu of options?**

Options: (a) List (b) Choice (c) Scrollbar (d) Menu

Answer: (b) Choice

## C. Code Output / Tracing Questions

**Q1. What is the output / result of this code snippet?**

Code:

```
Button b = new Button("Submit");  
b.setBounds(50, 80, 100, 40);  
frame.setLayout(null);  
frame.add(b);
```

Answer: A button labelled 'Submit' is displayed at position x=50, y=80  
with width=100px and height=40px. setLayout(null) enables manual positioning.

**Q2. What happens when the following listener is registered on a Frame?**

Code:

```
frame.addWindowListener(new WindowAdapter() {  
    public void windowClosing(WindowEvent e) {  
        System.exit(0);  
    }  
});
```

Answer: When the user clicks the X (close) button on the Frame,  
System.exit(0) is called, terminating the entire Java program.

**Q3. Identify the error in the following code:**

Code:

```
Checkbox c1 = new Checkbox("Option A");  
Checkbox c2 = new Checkbox("Option B");  
  
// Developer expects only one to be selected at a time
```



Answer: The developer forgot to use CheckboxGroup.

Without CheckboxGroup, both checkboxes can be selected simultaneously.

Fix: `CheckboxGroup grp = new CheckboxGroup();`

`Checkbox c1 = new Checkbox("Option A", false, grp);`

`Checkbox c2 = new Checkbox("Option B", false, grp);`

#### **Q4. What is the difference between these two statements?**

Statement 1: `frame.dispose();`

Statement 2: `System.exit(0);`

Answer: `frame.dispose()` only closes and frees resources of that specific Frame window.

`System.exit(0)` terminates the ENTIRE Java program/JVM immediately.

Use `dispose()` when you want to close a window but keep the program running.

## **D. Long Answer Questions (5-10 Marks)**

### **Q1. Explain the AWT hierarchy with the roles of Component, Container, and Layout Manager.**

**Component:** The base class for all visual elements (Button, Label, TextField, etc.).

Every visual thing in AWT extends Component.

**Container:** A special Component that can hold other Components.

Examples: Panel, Frame, Dialog, Window.

Containers can be nested (Panel inside Frame, Panel inside Panel).

**Layout Manager:** Controls how components are arranged inside a Container automatically.

**BorderLayout:** divides into North, South, East, West, Center regions.

**FlowLayout:** arranges left-to-right, top-to-bottom (default for Panel).

**GridLayout:** arranges in a fixed rows x columns grid.

`setLayout(null)`: disables auto layout; use `setBounds(x,y,w,h)` manually.

### **Q2. Describe all 5 methods of MouseListener with their triggering conditions.**

1. `mouseClicked(MouseEvent e)` — Fires after a complete click (press + release at same location).

2. `mousePressed(MouseEvent e)` — Fires as soon as the mouse button is held down.

3. `mouseReleased(MouseEvent e)` — Fires when the mouse button is let go.

4. `mouseEntered(MouseEvent e)` — Fires when the cursor moves INTO a component's boundary.

5. `mouseExited(MouseEvent e)` — Fires when the cursor moves OUT OF a component's boundary.

All 5 methods must be implemented when using MouseListener interface.

Use `MouseAdapter` (an adapter class) to avoid writing empty methods you don't need.

**Q3. Write a Java AWT program to create a simple login form with username (TextField), password (TextField), and a Login Button.**

```
import java.awt.*;
import java.awt.event.*;

public class LoginForm {

    public static void main(String[] args) {

        Frame f = new Frame("Login");

        Label l1 = new Label("Username:");
        Label l2 = new Label("Password:");
        TextField t1 = new TextField(20);
        TextField t2 = new TextField(20);
        t2.setEchoChar('*'); // Hide password

        Button btn = new Button("Login");

        l1.setBounds(50, 80, 80, 25); t1.setBounds(150, 80, 150, 25);
        l2.setBounds(50,120, 80, 25); t2.setBounds(150,120, 150, 25);
        btn.setBounds(150,170, 80, 30);

        f.add(l1); f.add(t1); f.add(l2); f.add(t2); f.add(btn);

        f.setSize(360, 260); f.setLayout(null); f.setVisible(true);

        btn.addActionListener(e -> System.out.println("Login clicked"));

        f.addWindowListener(new WindowAdapter() {

            public void windowClosing(WindowEvent e) { System.exit(0); }

        });

    }

}
```

## 8. Quick Reference Cheat Sheet

| Concept          | Key Point                                                                |
|------------------|--------------------------------------------------------------------------|
| AWT Full Form    | Abstract Window Toolkit                                                  |
| Package          | java.awt                                                                 |
| Component Type   | Heavy-weight, platform-dependent                                         |
| Main Container   | Frame (has title bar + border)                                           |
| Group Containers | Panel (lightweight, invisible grouper)                                   |
| Popup Window     | Dialog (temporary, needs parent frame)                                   |
| Button Click     | ActionListener → actionPerformed()                                       |
| Mouse Events     | MouseListener (5 methods) + MouseMotionListener (2 methods)              |
| Keyboard Events  | KeyListener → keyPressed, keyReleased, keyTyped                          |
| Window Events    | WindowListener → 7 methods (or use WindowAdapter)                        |
| Item Selection   | ItemListener → itemStateChanged()                                        |
| Radio Buttons    | CheckboxGroup + multiple Checkbox objects                                |
| Dropdown         | Choice class                                                             |
| Manual Position  | setLayout(null) + setBounds(x, y, w, h)                                  |
| Close Window     | dispose() — frees resources, keeps program alive                         |
| Exit Program     | System.exit(0) — terminates entire JVM                                   |
| Adapter Class    | Empty implementations of listener — extend & override only what you need |

■ *Good Luck on Your Exam! | Study Tip: Understand the hierarchy (Component → Container → Frame) and practise writing listener code from scratch.*

# AWT Login Page Java Code

```
import java.awt.*;
import java.awt.event.*;

public class LoginPage extends Frame implements ActionListener
{
    // Components
    Label title, userLabel, passLabel, message;
    TextField userText, passText;
    Button loginBtn, resetBtn;
    Checkbox remember;

    // Constructor
    LoginPage()
    {
        // Frame Title
        setTitle("AWT Login Page");

        // Layout
        setLayout(new FlowLayout());

        // Creating Components
        title = new Label("LOGIN FORM");

        userLabel = new Label("Username:");
        userText = new TextField(20);

        passLabel = new Label("Password:");
        passText = new TextField(20);
        passText.setEchoChar('*');

        remember = new Checkbox("Remember Me");

        loginBtn = new Button("Login");
        resetBtn = new Button("Reset");

        message = new Label("");

        // Adding Components
        add(title);

        add(userLabel);
        add(userText);

        add(passLabel);
        add(passText);

        add(remember);

        add(loginBtn);
        add(resetBtn);

        add(message);

        // Button Events
        loginBtn.addActionListener(this);
        resetBtn.addActionListener(this);

        // Frame Size
        setSize(300, 250);

        // Visible
        setVisible(true);

        // Closing Window
        addWindowListener(new WindowAdapter()
        {
            public void windowClosing(WindowEvent e)
            {
                dispose();
            }
        });
    }
    // Action Events
}
```

```

public void actionPerformed(ActionEvent e)
{
    if(e.getSource() == loginBtn)
    {
        String user = userText.getText();
        String pass = passText.getText();

        if(user.equals("admin") && pass.equals("1234"))
        {
            message.setText("Login Successful!");
        }
        else
        {
            message.setText("Invalid Username or Password");
        }
    }

    if(e.getSource() == resetBtn)
    {
        userText.setText("");
        passText.setText("");
        message.setText("");
    }
}

// Main Method
public static void main(String[] args)
{
    new LoginPage();
}
}

```